

✓ Analyse exploratoire du fichier population

```
import pandas as pd
import numpy as np
import math
```

```
population = pd.read_csv('population.csv')
dispo_alimentaire = pd.read_csv('dispo_alimentaire.csv')
aide_alimentaire = pd.read_csv('aide_alimentaire.csv')
sous_nutrition = pd.read_csv('sous_nutrition.csv')
```

```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(population.shape[0]))
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
```

```
↗ Le tableau comporte 1416 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)
```

[+ Code](#)[+ Texte](#)

```
print("population :", population.shape)
print("dispo_alimentaire :", dispo_alimentaire.shape)
print("aide_alimentaire :", aide_alimentaire.shape)
print("sous_nutrition :", sous_nutrition.shape)
```

```
↗ population : (1416, 3)
dispo_alimentaire : (15605, 18)
aide_alimentaire : (1475, 4)
sous_nutrition : (1218, 3)
```

```
population.count() #La nature des données dans chacune des colonnes
```

```
↗ Zone      1416
Année      1416
Valeur     1416
dtype: int64
```

```
print(population.dtypes) #Le nombre de valeurs présentes dans chacune des colonnes
```

```
↗ Zone      object
Année      int64
Valeur     float64
dtype: object
```

```
#Affichage les 5 premières lignes de la table
population.head(5) #Affichage les 5 premières lignes de la table
```



	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

```
#Multiplication de la colonne valeur par 1000
```

```
population['Valeur'] = population['Valeur'] * 1000
```

```
print(population)
```



	Zone	Année	Valeur
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0
...	...	...	...
1411	Zimbabwe	2014	13586707.0
1412	Zimbabwe	2015	13814629.0
1413	Zimbabwe	2016	14030331.0
1414	Zimbabwe	2017	14236595.0
1415	Zimbabwe	2018	14438802.0

```
[1416 rows x 3 columns]
```

```
#changement du nom de la colonne Valeur par Population
```

```
population.rename(columns={'Valeur':'Population'}, inplace=True)
```

```
population.head(5)
```



	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

```
#Affichage les 5 premières lignes de la table pour voir les modifications
```

```
population.head(5)
```



	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

✓ Analyse exploratoire du fichier disponibilité alimentaire

```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(dispo_alimentaire.shape[0]))
print("Le tableau comporte {} colonne(s)".format(dispo_alimentaire.shape[1]))
```



Le tableau comporte 15605 observation(s) ou article(s)
Le tableau comporte 18 colonne(s)

```
#Consulter le nombre de colonnes
print("dispo_alimentaire :", dispo_alimentaire.shape)
```



dispo_alimentaire : (15605, 18)

```
#Afficher les 5 premières lignes de la table
dispo_alimentaire.head(5)
```



	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/an)	Disponibilité de matière grasse en quantité (g/personne/jour)	Disponibilité protéique (g/personne/jour)
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	1.72	0.20	
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	1.29	0.01	
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	0.06	0.01	
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	0.00	NaN	
4	Afghanistan	Avocats	vegetale	NaN	NaN	1.0	0.70	0.00	

```
#remplacement des NaN dans le dataset par des 0
dispo_alimentaire.fillna(0, inplace=True)
```

```
#multiplication de toutes les lignes contenant des tonnes en Kg
colonnes_tonnes_tokg = ['Aliments pour animaux', 'Disponibilité intérieure', 'Exportations - Quantité',
                        'Importations - Quantité', 'Nourriture', 'Pertes', 'Production',
                        'Semences', 'Traitement', 'Variation de stock', 'Autres Utilisations']

for elt in colonnes_tonnes_tokg:
    dispo_alimentaire[elt] *= 1000000
```

```
#Afficher les 5 premières lignes de la table
dispo_alimentaire.head()
```



	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/an)	Disponibilité de matière grasse en quantité (g/personne/jour)	Disponibilité protéique en quantité (g/personne/jour)
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	1.72	0.20	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	1.29	0.01	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	0.06	0.01	
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	0.00	0.00	
4	Afghanistan	...	...	...	...	...	...	...	...



▼ Analyse exploratoire du fichier aide alimentaire

```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(aide_alimentaire.shape[0]))
print("Le tableau comporte {} colonne(s)".format(aide_alimentaire.shape[1]))
```



```
Le tableau comporte 1475 observation(s) ou article(s)
Le tableau comporte 4 colonne(s)
```

```
#Afficher les 5 premières lignes de la table
aide_alimentaire.head()
```



	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

```
#changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire.rename(columns={"Pays bénéficiaire": "Zone"}, inplace=True)
```

```
#changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire.rename(columns={"Aide_Alimentaire": "Valeur"}, inplace=True)
```

```
#Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000 pour avoir des kg
```

```
aide_alimentaire['Valeur'] = aide_alimentaire['Valeur'] * 1000
```

```
aide_alimentaire.head(5)
#1000kg : avoir les mêmes unités de mesure
```



	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

```
aide_alimentaire.head(5)
```



	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

✓ Analyse exploratoire du fichier sous nutrition

```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(sous_nutrition.shape[0]))
print("Le tableau comporte {} colonne(s)".format(sous_nutrition.shape[1]))
```



```
Le tableau comporte 1218 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)
```

```
#Consulter le nombre de colonnes
print(sous_nutrition.shape)
print(sous_nutrition.dtypes)
```

```
(1218, 3)
Zone      object
Année     object
Valeur    object
dtype: object
```

```
#Affichage les 5 premières lignes de la table
sous_nutrition.head(5)
```

```
Zone      Année  Valeur
0  Afghanistan  2012-2014    8.6
1  Afghanistan  2013-2015    8.8
2  Afghanistan  2014-2016    8.9
3  Afghanistan  2015-2017    9.7
4  Afghanistan  2016-2018   10.5
```

```
#Conversion de la colonne sous nutrition en numérique
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].replace('<0.1', '0.1')
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'])
sous_nutrition.head(5)
```

```
Zone      Année  Valeur
0  Afghanistan  2012-2014    8.6
1  Afghanistan  2013-2015    8.8
2  Afghanistan  2014-2016    8.9
3  Afghanistan  2015-2017    9.7
4  Afghanistan  2016-2018   10.5
```

```
#Conversion de la colonne (avec l'argument errors=coerce qui permet de convertir automatiquement les lignes qui ne sont pas des nombres en NaN)
#Puis remplacement des NaN en 0
sous_nutrition['Valeur'].fillna(0, inplace=True)
sous_nutrition.head(5)
```

```
Zone      Année  Valeur
0  Afghanistan  2012-2014    8.6
1  Afghanistan  2013-2015    8.8
2  Afghanistan  2014-2016    8.9
3  Afghanistan  2015-2017    9.7
4  Afghanistan  2016-2018   10.5
```

```
#changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition.rename(columns={'Valeur':'sous_nutrition'}, inplace=True)
sous_nutrition.head()
```



	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
#Multiplication de la colonne sous_nutrition par 1000000
sous_nutrition['sous_nutrition'] = sous_nutrition['sous_nutrition'] * 1000000
sous_nutrition.head()
```



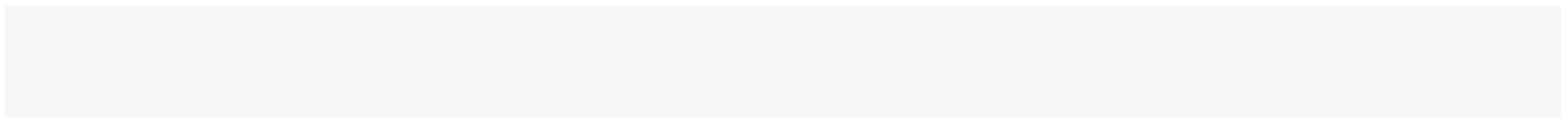
	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

```
#Afficher les 5 premières lignes de la table
sous_nutrition.head(5)
```



	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

✓ Proportion de personnes en sous nutrition



```
sous_nutrition_2017 = sous_nutrition[sous_nutrition['Année'] == '2016-2018']
population_2017 = population[population['Année'] == 2017]
jointure_2017 = pd.merge(population_2017, sous_nutrition_2017, on = 'Zone')
```

```
jointure_2017.head(10)
```



	Zone	Année_x	Population	Année_y	sous_nutrition
0	Afghanistan	2017	36296113.0	2016-2018	10500000.0
1	Afrique du Sud	2017	57009756.0	2016-2018	3100000.0
2	Albanie	2017	2884169.0	2016-2018	100000.0
3	Algérie	2017	41389189.0	2016-2018	1300000.0
4	Allemagne	2017	82658409.0	2016-2018	0.0
5	Andorre	2017	77001.0	2016-2018	0.0
6	Angola	2017	29816766.0	2016-2018	5800000.0
7	Antigua-et-Barbuda	2017	95426.0	2016-2018	0.0
8	Arabie saoudite	2017	33101179.0	2016-2018	1600000.0
9	Argentine	2017	43937140.0	2016-2018	1500000.0

```
jointure_2017 = pd.merge(population.loc[population['Année'] == 2017,["Zone", "Population"]],
                        sous_nutrition.loc[sous_nutrition['Année'] == '2016-2018',["Zone", "sous_nutrition"]],
                        on='Zone')
```

```
jointure_2017.head(10)
```



	Zone	Population	sous_nutrition
0	Afghanistan	36296113.0	10500000.0
1	Afrique du Sud	57009756.0	3100000.0
2	Albanie	2884169.0	100000.0
3	Algérie	41389189.0	1300000.0
4	Allemagne	82658409.0	0.0
5	Andorre	77001.0	0.0
6	Angola	29816766.0	5800000.0
7	Antigua-et-Barbuda	95426.0	0.0
8	Arabie saoudite	33101179.0	1600000.0
9	Argentine	43937140.0	1500000.0


```
#Affichage du nombre de personnes en état de sous nutrition
print("Proportion de personnes en état de sous nutrition :", "{:.2f}".format(jointure_2017['sous_nutrition'].sum()*100/jointure_2017['Population'].sum()), "%")
```

Proportion de personnes en état de sous nutrition : 7.13 %

```
#Affichage du dataset
print(jointure_2017.dtypes)
```

Zone object
Population float64
sous_nutrition float64
dtype: object

```
#Calcul et affichage du nombre de personnes en état de sous nutrition
filtered_df = jointure_2017[jointure_2017['sous_nutrition'] == 0]
total_population = filtered_df['Population'].sum()

print("Total de la population où sous_nutrition est égal à 0:", total_population)
```

Total de la population où sous_nutrition est égal à 0: 3360967421.0

ici # Nombre théorique de personne qui pourrait être nourries

Combien mange en moyenne un être humain ? Source => 2615 Kilo calories par jour
https://www.google.com/search?q=Combien+mange+en+moyenne+un+C3%Aatre+humain+dans+le+monde&oq=Combien+mange+en+moyenne+un+C3%Aatre+humain+dans+le+monde&gs_lcrp=EgZjaHJvbWUqBggAEEUYOzIGCAAQRrg7MgYIAF

On commence par faire une jointure entre le data frame population et Dispo_alimentaire afin d'ajouter dans ce dernier la population

```
population_2017 = population[population['Année'] == 2017]
jointure1 = pd.merge(population_2017, dispo_alimentaire, on='Zone', how='right')
jointure1.head(5)
```

	Zone	Année	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/an)	Disponibi matière gr q (g/personn
0	Afghanistan	2017.0	36296113.0	Abats Comestible	animale	0.0	0.0	5.0	1.72	
1	Afghanistan	2017.0	36296113.0	Agrumes, Autres	vegetale	0.0	0.0	1.0	1.29	
2	Afghanistan	2017.0	36296113.0	Aliments pour enfants	vegetale	0.0	0.0	1.0	0.06	
3	Afghanistan	2017.0	36296113.0	Ananas	vegetale	0.0	0.0	0.0	0.00	
4	Afghanistan	2017.0	36296113.0	Bananes	vegetale	0.0	0.0	4.0	2.70	

```
#Affichage du nouveau dataframe
print(jointure1.dtypes)
```

Zone	object
Année	float64
Population	float64
Produit	object
Origine	object
Aliments pour animaux	float64
Autres Utilisations	float64
Disponibilité alimentaire (Kcal/personne/jour)	float64
Disponibilité alimentaire en quantité (kg/personne/an)	float64
Disponibilité de matière grasse en quantité (g/personne/jour)	float64
Disponibilité de protéines en quantité (g/personne/jour)	float64
Disponibilité intérieure	float64
Exportations - Quantité	float64
Importations - Quantité	float64
Nourriture	float64
Pertes	float64
Production	float64
Semences	float64
Traitement	float64
Variation de stock	float64
dtype:	object

```
jointure1["dispo_kcal"] = jointure1["Population"] * jointure1["Disponibilité alimentaire (Kcal/personne/jour)"]
jointure1.head()
```

	Zone	Année	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/an)	Disponibi matière gr q
0	Afghanistan	2017.0	36296113.0	Abats Comestible	animale	0.0	0.0	5.0	1.72	
1	Afghanistan	2017.0	36296113.0	Agrumes, Autres	vegetale	0.0	0.0	1.0	1.29	
2	Afghanistan	2017.0	36296113.0	Aliments pour enfants	vegetale	0.0	0.0	1.0	0.06	
3	Afghanistan	2017.0	36296113.0	Ananas	vegetale	0.0	0.0	0.0	0.00	
4	Afghanistan	2017.0	36296113.0	Bananes	vegetale	0.0	0.0	4.0	2.70	

```
#Création de la colonne dispo_kcal et calcul des kcal disponibles mondialement
jointure1['dispo_kcal'] = jointure1['Disponibilité alimentaire (Kcal/personne/jour)'] * jointure1['Population'] * 365
print("dispo alimentaire totale en kcal :", jointure1['dispo_kcal'].sum(), "kcal")
```

```
dispo alimentaire totale en kcal : 7637384005750065.0 kcal
```

```
#Calcul du nombre d'humain pouvant être nourris
total_h_kcal = round(jointure1['dispo_kcal'].sum()/(2250*365))
print("Total d'être humain pouvant être nourris :", total_h_kcal)
print("Proportion :", "{:.2f}".format(total_h_kcal*100/population.loc[population['Année'] == 2017,"Population"].sum()), "%")
```

```
➡ Total d'être humain pouvant être nourris : 9299706552
Proportion : 123.21 %
```

✓ Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

```
#Transfert des données avec les végétaux dans un nouveau dataframe
Origine_vegetale = jointure1[jointure1['Origine'].str.contains('vegetale', case=False, na=False)]
Origine_vegetale["humain_nourris"] = Origine_vegetale["Population"] * Origine_vegetale["Disponibilité alimentaire (Kcal/personne/jour)"]

Origine_vegetale.head(5)
```

```
➡ <ipython-input-165-c4e55342c982>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
Origine_vegetale["humain_nourris"] = Origine_vegetale["Population"] * Origine_vegetale["Disponibilité alimentaire (Kcal/persor
```

	Zone	Année	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/an)	Disponibilité en matière grasse (g/personne, qu)
1	Afghanistan	2017.0	36296113.0	Agrumes, Autres	vegetale	0.0	0.0	1.0	1.29	
2	Afghanistan	2017.0	36296113.0	Aliments pour enfants	vegetale	0.0	0.0	1.0	0.06	
3	Afghanistan	2017.0	36296113.0	Ananas	vegetale	0.0	0.0	0.0	0.00	
4	Afghanistan	2017.0	36296113.0	Bananes	vegetale	0.0	0.0	4.0	2.70	
6	Afghanistan	2017.0	36296113.0	Bière	vegetale	0.0	0.0	0.0	0.09	

5 rows x 11 columns

```
#Calcul du nombre de kcal disponible pour les vegetaux
print("dispo alimentaire totale en kcal des produits végétaux :", Origine_vegetale['dispo_kcal'].sum(), "kcal")
```

```
➡ dispo alimentaire totale en kcal des produits végétaux : 6302133553972115.0 kcal
```

```
#Calcul du nombre d'humain pouvant être nourris avec les vegetaux
total_h_kcal = round(Origine_vegetale['dispo_kcal'].sum()/(2250*365))
print("Total d'être humain pouvant être nourris :", total_h_kcal)
print("Proportion :", "{:.2f}".format(total_h_kcal*100/population.loc[population['Année'] == 2017,"Population"].sum()), "%")
```

```
➦ Total d'être humain pouvant être nourris : 7673830812
Proportion : 101.67 %
```

✓ Utilisation de la disponibilité intérieure

```
#Calcul de la disponibilité totale
total_disponibilite_interieure = dispo_alimentaire['Disponibilité intérieure'].sum()

print("La somme totale des valeurs de la colonne 'Disponibilité intérieure' est:", total_disponibilite_interieure)
```

```
➦ La somme totale des valeurs de la colonne 'Disponibilité intérieure' est: 9848994000000.0
```

```
#création d'une boucle for pour afficher les différentes valeurs en fonction des colonnes aliments pour animaux, pertes, nourritures,
```

```
total_nourriture_animaux = jointure1["Aliments pour animaux"].sum()
total_pertes = jointure1["Pertes"].sum()
total_nourriture = jointure1["Nourriture"].sum()
```

```
print("Nourriture pour animaux totale :", total_nourriture_animaux)
print("Pertes totales :", total_pertes)
print("Nourriture totale :", total_nourriture)
```

```
➦ Nourriture pour animaux totale : 1304245000000.0
Pertes totales : 453698000000.0
Nourriture totale : 4876258000000.0
```

```
#création d'une boucle for pour afficher les différentes valeurs en fonction des colonnes aliments pour anivaux, pertes, nourritures,
for elt in ['Aliments pour animaux', 'Pertes', 'Nourriture', 'Semences', 'Traitement', 'Autres Utilisations']:
    print("Proportion de", elt, ":", "{:.2f}".format(dispo_alimentaire[elt].sum()*100/total_disponibilite_interieure), "%")
```

```
➦ Proportion de Aliments pour animaux : 13.24 %
Proportion de Pertes : 4.61 %
Proportion de Nourriture : 49.51 %
Proportion de Semences : 1.57 %
Proportion de Traitement : 22.38 %
Proportion de Autres Utilisations : 8.78 %
```

```
total_nourriture_animaux = jointure1["Aliments pour animaux"].sum()
total_pertes = jointure1["Pertes"].sum()
total_nourriture = jointure1["Nourriture"].sum()
total_semences = dispo_alimentaire['Semences'].sum()
total_traitement = dispo_alimentaire['Traitement'].sum()
total_autres_utilisations = dispo_alimentaire['Autres Utilisations'].sum()
```

```
print("Nourriture pour animaux totale :", total_nourriture_animaux, "kg")
print("Pertes totales :", total_pertes, "kg")
print("Nourriture totale :", total_nourriture, "kg")
print("Semences totales :", total_semences, "kg")
print("Traitement total :", total_traitement, "kg")
print("Autres utilisations totales :", total_autres_utilisations, "kg")
```

```
➦ Nourriture pour animaux totale : 1304245000000.0 kg
Pertes totales : 453698000000.0 kg
Nourriture totale : 4876258000000.0 kg
Semences totales : 154681.0 kg
Traitement total : 2204687.0 kg
Autres utilisations totales : 865023.0 kg
```

```
#Création d'une liste avec toutes les variables
liste_cereale = 'Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Seigle', 'Avoine', 'Millet', 'Sorgho', 'Céréales'

print(liste_cereale)
```

```
➦ ('Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Seigle', 'Avoine', 'Millet', 'Sorgho', 'Céréales')
```

```
#Création d'un dataframe avec les informations uniquement pour ces céréales
#print(aide_alimentaire)

liste_cereale = 'Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Seigle', 'Avoine', 'Millet', 'Sorgho', 'Céréales'
cereales_df = dispo_alimentaire.loc[dispo_alimentaire['Produit'].isin(liste_cereale),:]

cereales_df.head()
```

```
➦
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/an)	Disponibilité de matière grasse en quantité (g/personne/jour)	Disponib prot (g/person
7	Afghanistan	Blé	vegetale	0.0	0.0	1369.0	160.23	4.69	
32	Afghanistan	Maïs	vegetale	200000000.0	0.0	21.0	2.50	0.30	
34	Afghanistan	Millet	vegetale	0.0	0.0	3.0	0.40	0.02	
40	Afghanistan	Orge	vegetale	360000000.0	0.0	26.0	2.92	0.24	
47	Afghanistan	Riz (Eq	vegetale	0.0	0.0	141.0	13.82	0.27	

```
#Affichage de la proportion d'alimentation animale
print("Proportion d'alimentation animale :", "{:.2f}".format(cereales_df['Aliments pour animaux'].sum()*100/cereales_df['Disponibilité intérieure'].sum()), "%")
```

```
➦ Proportion d'alimentation animale : 35.91 %
```

```
#Affichage de la proportion d'alimentation humaine
print("Proportion d'alimentation humaine :", "{:.2f}".format(cereales_df['Nourriture'].sum()*100/cereales_df['Disponibilité intérieure'].sum()), "%")
```

```
➦ Proportion d'alimentation humaine : 43.02 %
```

✓ Ici Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

```
#Création de la colonne proportion par pays sous nutrition/pop,3.1
```

```
def reformat(annee):  
    debut, fin = map(int, annee.split('-'))  
    return (debut + fin) // 2  
sous_nutrition['Année'] = sous_nutrition['Année'].apply(reformat)  
  
sous_nutrition_2017 = sous_nutrition[sous_nutrition['Année'] == 2017]  
#population_2017 = population[population['Année'] == 2017]  
#jointure_2017 = pd.merge(population_2017, sous_nutrition_2017, on = 'Zone')
```

```
sous_nutrition_2017.head()
```



	Zone	Année	sous_nutrition
4	Afghanistan	2017	10500000.0
10	Afrique du Sud	2017	3100000.0
16	Albanie	2017	100000.0
22	Algérie	2017	1300000.0
28	Allemagne	2017	0.0

```
jointure_2017['proportion'] =jointure_2017['sous_nutrition']/jointure_2017['Population']  
jointure_2017.head()
```



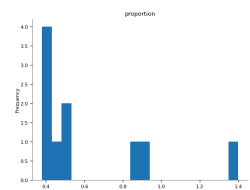
	Zone	Population	sous_nutrition	proportion
0	Afghanistan	36296113.0	10500000.0	0.289287
1	Afrique du Sud	57009756.0	3100000.0	0.054377
2	Albanie	2884169.0	100000.0	0.034672
3	Algérie	41389189.0	1300000.0	0.031409
4	Allemagne	82658409.0	0.0	0.000000

```
#affichage après trie des 10 pires pays ajouter le volume sous nutrition  
jointure_2017[['Zone', "proportion"]].sort_values(by="proportion", ascending=False).head(10)
```

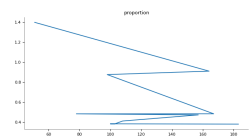


	Zone	proportion
51	Dominique	1.399423
164	Saint-Vincent-et-les Grenadines	0.910523
98	Kiribati	0.875979
167	Sao Tomé-et-Principe	0.482884
78	Haïti	0.482592
157	République populaire démocratique de Corée	0.471887
108	Madagascar	0.410629
103	Libéria	0.382797
100	Lesotho	0.382494
183	Tchad	0.379576

Distributions



Values



```
import pandas as pd
# Tri des données et extraction des 10 premiers pays
top_10 = jointure_2017.sort_values(by="proportion", ascending=False).head(10)

# Création du tableau
table = pd.DataFrame({
    'Pays': top_10['Zone'],
    'Proportion de personnes sous-alimentées': top_10['proportion']
})

# Affichage du tableau
print(table)
```



	Pays \
51	Dominique
164	Saint-Vincent-et-les Grenadines
98	Kiribati
167	Sao Tomé-et-Principe
78	Haïti

157	République populaire démocratique de Corée
108	Madagascar
103	Libéria
100	Lesotho
183	Tchad

Proportion de personnes sous-alimentées	
51	1.399423
164	0.910523
98	0.875979
167	0.482884
78	0.482592
157	0.471887
108	0.410629
103	0.382797
100	0.382494
183	0.379576

✓ Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

```
#calcul du total de l'aide alimentaire par pays

somme_par_pays= aide_alimentaire[['Pays bénéficiaire', 'Valeur']].groupby("Pays bénéficiaire").sum().reset_index()

somme_par_pays.head(10)
```

↕

	Pays bénéficiaire	Valeur
0	Afghanistan	185452
1	Algérie	81114
2	Angola	5014
3	Bangladesh	348188
4	Bhoutan	2666
5	Bolivia (État plurinational de)	6
6	Burkina Faso	64812
7	Burundi	77318
8	Bénin	22224
9	Cambodge	25780

```
#affichage après trie des 10 pays qui ont bénéficié le plus de l'aide alimentaire
ten_worst_aa = somme_par_pays['Valeur'].nlargest(10)
# Get the index values corresponding to the top 10 values
ten_worst_index_aa = ten_worst_aa.index
ten_worst_countries_aa = somme_par_pays.loc[ten_worst_index_aa, ['Zone', 'Valeur']]

ten_worst_countries_aa.head(10)
```




	Zone	Valeur
50	République arabe syrienne	1858943000
75	Éthiopie	1381294000
70	Yémen	1206484000
61	Soudan du Sud	695248000
60	Soudan	669784000
30	Kenya	552836000
3	Bangladesh	348188000
59	Somalie	292678000
53	République démocratique du Congo	288502000
43	Niger	276344000

✓ Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

```
#Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis groupby sur zone et année
```

```
evolution_2013_2016 = aide_alimentaire.groupby(['Zone', 'Année'])['Valeur'].sum()
```

```
print(evolution_2013_2016)
```



Zone	Année	
Afghanistan	2013	128238000
	2014	57214000
Algérie	2013	35234000
	2014	18980000
	2015	17424000
...		
Égypte	2013	1122000
Équateur	2013	1362000
Éthiopie	2013	591404000
	2014	586624000
	2015	203266000
Name: Valeur, Length: 228, dtype: int64		

```
#affichage après trie des 10 pays qui ont bénéficié le plus de l'aide alimentaire
```

```
somme_par_pays = aide_alimentaire.groupby('Zone')['Valeur'].sum()
```

```
top_10_pays = somme_par_pays.nlargest(10)
```

```
print(top_10_pays)
```



Zone	
République arabe syrienne	1858943000
Éthiopie	1381294000
Yémen	1206484000
Soudan du Sud	695248000

```
Soudan                669784000
Kenya                  552836000
Bangladesh             348188000
Somalie                292678000
République démocratique du Congo  288502000
Niger                  276344000
Name: Valeur, dtype: int64
```

✓ Pays avec le moins de disponibilité par habitant

```
#Calcul de la disponibilité en kcal par personne par jour par pays
```

```
result1 = jointure1.groupby(['Zone', 'Produit'])['Disponibilité alimentaire (Kcal/personne/jour)'].sum().reset_index()
```

```
result1
```



	Zone	Produit	Disponibilité alimentaire (Kcal/personne/jour)
0	Afghanistan	Abats Comestible	5.0
1	Afghanistan	Agrumes, Autres	1.0
2	Afghanistan	Aliments pour enfants	1.0
3	Afghanistan	Ananas	0.0
4	Afghanistan	Bananes	4.0
...	...	...	...
15600	Îles Salomon	Viande de Suides	45.0
15601	Îles Salomon	Viande de Volailles	11.0
15602	Îles Salomon	Viande, Autre	0.0
15603	Îles Salomon	Vin	0.0
15604	Îles Salomon	Épices, Autres	4.0

15605 rows × 3 columns

```
#Affichage des 10 pays qui ont le moins de dispo alimentaire par personne
```

```
somme1 = result1.groupby('Zone')['Disponibilité alimentaire (Kcal/personne/jour)'].sum().reset_index()
```

```
filtre_value = somme1.sort_values(by='Disponibilité alimentaire (Kcal/personne/jour)')
```

```
top_10_lowest = filtre_value.head(10)
```

```
top_10_lowest
```



	Zone	Disponibilité alimentaire (Kcal/personne/jour)
128	République centrafricaine	1879.0
166	Zambie	1924.0
91	Madagascar	2056.0
0	Afghanistan	2087.0
65	Haïti	2089.0
133	République populaire démocratique de Corée	2093.0
151	Tchad	2109.0
167	Zimbabwe	2113.0
114	Ouganda	2126.0
154	Timor-Leste	2129.0

✓ Pays avec le plus de disponibilité par habitant

```
#Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
filtre_value = somme1.sort_values(by='Disponibilité alimentaire (Kcal/personne/jour)', ascending=False)
top_10_highest = filtre_value.head(10)

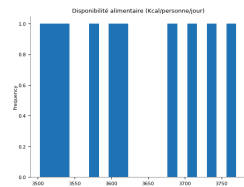
top_10_highest
```



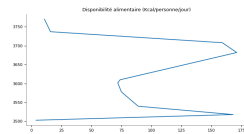
Zone Disponibilité alimentaire (Kcal/personne/jour)

11	Autriche	3770.0
16	Belgique	3737.0
159	Turquie	3708.0
171	États-Unis d'Amérique	3682.0
74	Israël	3610.0
72	Irlande	3602.0
75	Italie	3578.0
89	Luxembourg	3540.0
168	Égypte	3518.0
4	Allemagne	3503.0

Distributions



Values



✓ Exemple de la Thaïlande pour le Manioc

#création d'un dataframe avec uniquement la Thaïlande

```
#print(population) Pays bénéficiaire 2017
#print(dispo_alimentaire) Pays bénéficiaire
#print(aide_alimentaire) #Pays bénéficiaire 2017
print(sous_nutrition) #zone 2017
```



	Zone	Année	sous_nutrition
0	Afghanistan	2013	8600000.0
1	Afghanistan	2014	8800000.0
2	Afghanistan	2015	8900000.0
3	Afghanistan	2016	9700000.0
4	Afghanistan	2017	10500000.0
...	...	...	...
1213	Zimbabwe	2014	0.0

1214	Zimbabwe	2015	0.0
1215	Zimbabwe	2016	0.0
1216	Zimbabwe	2017	0.0
1217	Zimbabwe	2018	0.0

[1218 rows x 3 columns]

```
sous_nutrition_Thailande_2017 = jointure_2017[jointure_2017['Zone'] == 'Thaïlande']
```

```
sous nutrition Thaïlande 2017
```